

THE INFOPHARMA ORDERING SYSTEM (IPOS) SOFTWARE IPOS – PU

Centre for Software Reliability,
iOrder
City St George's, University of London
London, 2026

SUMMARY

This is a Requirements Specification and High-Level Design Document, a proposal to the Managing Director of Info Pharma Ltd, by Dr Peter Popov, Reader of Systems Dependability at the Centre for Software Reliability, City St George's, University of London.

AUTHORS

Ethan Kan, Joyce Ow, Sofya Shorunova, Seyer Raji, Manasar Abdullahi, Basit Hussain

DISTRIBUTION

The Managing Director of Info Pharma Ltd, the Development Team at the Centre for Software Reliability at iOrder, City St George's, University of London.



CONTENTS

1. Preface	4
1.1 Purpose and scope of the document (what it is and what is NOT)	4
1.2 Intended audience and Reading suggestions	4
1.3 History of the document.....	4
2. introduction	4
2.1 Scope of this document	5
2.2 Intended Audience	6
3. Glossary	6
4. User requirements.....	6
5. System requirements.....	6
5.1 Purpose and scope of the system.....	6
5.2 System models.....	7
5.3 UML use-case models.....	7
5.4 Use case diagram.....	7
5.2.1.2 Use case specification (simplified).....	8
5.5 Prioritised List of use cases.....	27
6. Anticipated evolution of system requirements.....	29
6.1 Customer Registration and Verification	29
6.2 Payment Processing.....	29
6.3 Promotional Campaigns	30
6.4 Customer Engagement Tracking.....	30
6.5 Catalogue and Inventory Synchronisation.....	30
6.6 Customer Loyalty and Retention	31
7. IPOS High – Level Design	31
7.1 IPOS – SA.....	31
7.2 IPOS – CA	31
7.3 IPOS – PU	31
7.4 Software architecture	31
7.4. Design diagram of subsystem IPOS-PU.....	33
7.5 Source code (Java) for the interfaces of IPOS-PU.....	35

7.6 Testing the communications between subsystems 13.....	38
8. Appendices	45
8.1 Initial Statement of requirements by the client	45
IPOS: InfoPharma Ordering System	45
8.2 Hardware (minimal and recommended configuration)	46
8.3 Minimal hardware configuration.....	46
8.4 Recommended hardware configuration	47
9. Index	47

1. PREFACE

1.1 Purpose and scope of the document (what it is and what is NOT)

This is a Requirements Specification for one of the subsystems of The InfoPharma Ordering System (IPOS), IPOS-PU, and High-Level Design document for the IPOS software, which includes IPOS-PU and two other subsystems, IPOS-SA and IPOS-CA. The document covers the functional requirements of IPOS-PU and the high-level design of the entire IPOS system showing the IPOS architecture composed of the three subsystems and the mechanisms of interaction between them. The document is provided in response to the initial statement of requirements by Info Pharma Ltd, the Client.

The document explains what the subsystem shall do, and how its subsystems might interact with one another, but does not cover details about how the functionality of IPOS-PU subsystem will be implemented. Various UML models are used together with fragments of software code to convey in more detail to system developers the expected behaviour of the IPOS-PU subsystem and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended audience and Reading suggestions

The Managing Director of Info Pharma Ltd, the system analysts and architects at the Centre for Software Reliability (CSR), iOrder, City St George's, University of London.

Some of the sections are intended for the CSR team only, which is clearly stated in the document

1.3 History of the document

This is a new document, v.1.0.

2. INTRODUCTION

The IPOS-PU subsystem will provide an online portal through which a merchant can sell pharmaceutical goods directly to members of the public, and will support customer registration, online ordering, payment processing, order tracking, and promotional campaigns. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

IPOS-PU is one of three cooperating IPOS subsystems and focuses on the merchant's public storefront and marketing capabilities. It will integrate with the merchant's local point-of-sale and inventory (IPOS-CA) so that successful online purchases decrement the merchant's stock, and with the central catalogue and account services (IPOS-SA) for product data, pricing, and merchant membership verification. The portal will also record campaign engagement metrics to help merchants measure conversion and campaign effectiveness.

2.1 Scope of this document

Primary focus will be the requirements and high-level design for IPOS-PU only. The following will be included: functional requirements for public membership and sales, promotional campaigns management, basic payment and email communication flows, reporting needs for sales and campaign engagement, and the interfaces IPOS-PU must expose/consume to interoperate with IPOS-CA and IPOS-SA. Altogether, some of the things that will be excluded, will be the detailed implementation of InfoPharma's back-office accounting and full payment-gateway certification; these are treated as external services or optional integrations and are described only to the level required for interface design.

2.2 Intended Audience

- **Merchants and merchant administrators** – who will use the portal to sell and promote goods
- **InfoPharma stakeholders** – responsible for catalogue, merchant onboarding, and operations
- **Development teams** – responsible for implementing IPOS-PU and for defining the interfaces to IPOS-CA and IPOS-SA
- **Testers and QA** – who will validate functional scenarios, campaign tracking, and cross-subsystem interactions

3. GLOSSARY

Actor	A human or an external system interacting with IPOS, e.g. Customer, staff member, SMTP server, etc.
CSR	Centre for Software Reliability
IPOS	InfoPharma Ordering System
UML	Unified Modelling Language
Use case	A basic interaction of a system with an actor

4. USER REQUIREMENTS

This section is left intentionally blank¹.

5. SYSTEM REQUIREMENTS

5.1 Purpose and scope of the system

This section provides a more detailed specification of required functionality (known as system requirements) for IPOS-PU .

¹ A complete Requirements specification will include User (i.e. high level) requirements which do not require expertise in software engineering, e.g. using a popular form of recording user requirements such as Volere Template. The purpose of this document is different – to provide a template for the 1st Team deliverable in IN2033. To reduce the effort expected from students some parts of analysis and design are omitted, user requirements are one of these parts.

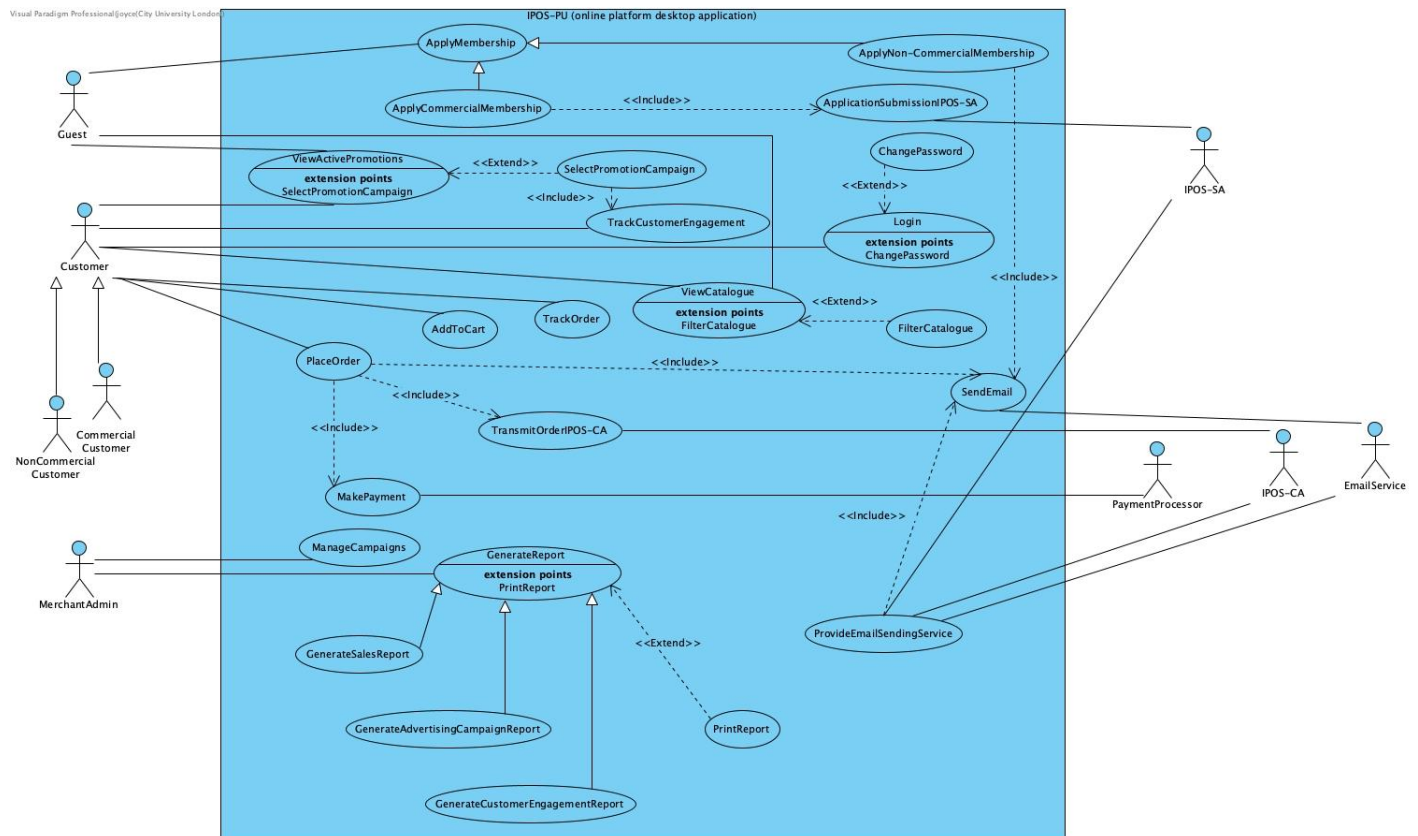
The intended audience are the *system architects, designers and software developers* responsible for the implementation of IPOS. Since we use UML, which requires a degree of specific expert knowledge, we do not expect this section to be immediately useful to the Info Pharma Ltd staff.

5.2 System models

5.2.1 UML use-case models²

5.2.1.1 Use case diagram

The use case diagram for IPOS-PU subsystem is shown below in Figure 1. In this diagram the other subsystems of IPOS are treated as “external systems”, with which IPOS-PU may interact.



² The use case model (use case diagram and specifications) can be replaced by a set of “user stories”.

Figure 1. Use case diagram of IPOS-PU. The use cases cover the required functionality (see the initial statement of requirements). The actors represent the external entities (users or 3rd party systems) which interact with IPOS-PU. The other subsystems of IPOS are also modelled as actors.

5.2.1.2 Use case specification (simplified)

A sample of **simplified** use case specification is provided³

Use case ID: 1	Use case: ApplyMembership
Brief Description: Allows a guest to submit an application for a membership account (commercial or non-commercial).	
Primary actors: Guest	
Secondary actors: None	
Preconditions: 1. IPOS-PU portal is accessible 2. Guest can select the option to register/apply for membership	
Main flow: Not required	
Extension use cases: PrintReport (if a confirmation/receipt is printed)	
Inclusion use cases: None	
Specialisations of: None	
Postconditions: 1. Guest's application has been submitted 2. Confirmation is presented to guest	
Alternative flows: • Guest cancels application – no data stored; guest returns to homepage	

³ The 1st Team deliverable of IN2033 should include a **complete** set of simplified specifications for all use cases. The simplification is in terms of not providing detailed flows (main and alternative) of interaction between the actors and the systems

- System error during submission - the guest is informed and prompted to retry

Use case ID: 2	Use case: ApplyCommercialMembership
Brief Description: Allows a company representative to submit an application for commercial membership, including company registration number and director details.	
Primary actors: Guest	
Secondary actors: IPOS-SA	
Preconditions: 1. Actor is accessing IPOS-PU as guest (not logged in) 2. Actor selects 'Commercial' membership type during application process	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: Apply 1. ApplicationSubmissionIPOS-SA (UC3) 2. ApplyMembership (UC1)	
Specialisation of: ApplyMembership (UC1)	
Postconditions: Application data is transmitted to IPOS-SA for diligence checks/pending staff review. Applicant receives confirmation notification	
Alternative flows: InvalidRegistrationNumber IPOS-SAConnectionFailure.	

Use case ID: 3	Use case: ApplicationSubmissionIPOS-SA
Brief Description: Transmits commercial membership application data from IPOS-PU to the IPOS-SA subsystem for review and approval by InfoPharma Ltd staff	
Primary actors: IPOS-SA	
Secondary actors: MerchantAdmin	
Preconditions: 1. Application form submitted by guest through ApplyCommercialMembership 2. Application data (company registration number, director details, business type, address, email) = complete and valid 3. IPOS-PU has connectivity to IPOS-SA	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: SendEmail	
Specialisations of: None	
Postconditions: Data successfully received by IPOS-SA and stored, awaiting review Applicant notified their application has been received and is under review	
Alternative flows: IPOS-SA Unavailable – system queue Transmission fails after retries – logs error by system and notifies MerchantAdmin for manual process forward application	

Use case ID: 4	Use case: ApplyNon-CommercialMembership
Brief Description: Allows a guest to register for a non-commercial account by providing an email address. The system validates the email, generates credentials, and sends it to the customer.	

Primary actors: Guest
Secondary actors: None
Preconditions: 1. IPOS-PU portal is accessible 2. Selects 'Non-Commercial' membership type (from ApplyMembership)
Main flow: Not required
Extension use cases: ChangePassword (extended Login for first time)
Inclusion use cases: SendEmail (UC4), ApplyMembership (UC1)
Specialisation of: ApplyMembership (UC1)
Postconditions: 1. A new non-commercial account is created, active status. 2. Login credentials are sent to the customer. 3. The customer is flagged to change their password upon their first login. 4. Order counter initialised to track 10th order discount (10% discount on every 10th order)
Alternative flows: InvalidEmailFormat EmailAddressAlreadyRegistered EmailServiceConnectionFailure

Use case ID: 5	Use case: SendEmail
Brief Description: Sends automated emails for registrations, orders, and tracking/payment reminders via a third-party SMTP service.	
Primary actors: IPOS-PU	
Secondary actors:	

EmailService
Preconditions: 1. Trigger event (registration, order placement, payment confirmation) 2. Recipient's email address known 3. IPOS-PU-COMMS configured and available
Main flow: Not required
Extension use cases: None
Inclusion use cases: ProvideEmailSendingService
Specialisations of: None.
Postconditions: Email is sent, record stored in the database.
Alternative flows: SMTPServiceFailure

Use case ID: 6	Use case: Login
Brief Description: Allows registered member (NonCommercial or Commercial Customer) to authenticate into IPOS-PU using username and password	
Primary actors: Customer (NonCommercial or Commercial)	
Secondary actors: None.	
Preconditions: 1. Actor has registered and have active account on IPOS-PU 2. Actor is on IPOS-PU login page	
Main flow: Not required	
Extension use cases:	

1. ChangePassword (UC7)
Inclusion use cases: None
Specialisations of: None
Postconditions: 1. User is authenticated and granted access. 2. Session established
Alternative flows: IncorrectCredentials AccountSuspended or inactive CustomerForgotPassword

Use case ID: 7	Use case: ChangePassword
Brief Description: Allows a customer to update their current password. This is a mandatory requirement for non-commercial customers during their first login session.	
Primary actors: Customer (NonCommercial or Commercial)	
Secondary actors: None	
Preconditions: 1. The customer is successfully authenticated or is in the process of a first-time login using a system-generated temporary password 2. Flags account to require password change on first login	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Specialisations of: None	
Postconditions:	

1. The new password is saved in the database, and the temporary password (if applicable) is invalidated
Alternative flows: NewPasswordCriteriaNotMet PasswordConfirmationAndNewPasswordNotMatched SystemTimeout DatabaseConnectionError

Use case ID: 8	Use case: ViewCatalogue
Brief Description: Allows the customer to browse the full list of pharmaceutical products offered by the merchant on the IPOS-PU online portal. Option to filter results and add items to their shopping cart	
Primary actors: Customer (NonCommercial or Commercial)	
Secondary actors: None	
Preconditions: 1. Customer logged in to IPOS-PU 2. Merchant's stock catalogue contains min one product	
Main flow: Not required	
Extension use cases: 1. FilterCatalogue (UC9) 2. AddToCart	
Inclusion use cases: None	
Specialisations of: None	
Postconditions: 1. The system displays the catalogue containing Item IDs, descriptions, unit prices, and current availability 2. Customer may have applied filters and/or added items to shopping cart	

Alternative flows:

EmptyCatalogue

If product has zero availability – system displays but no ‘Add To Cart’ option

Customer applies filter, returns no results – system displays ‘no products found’ message

Use case ID: 9	Use case: FilterCatalogue
Brief Description: Allows the customer to narrow down the list of products in the catalogue through specific keywords such as "paracetamol", Item ID, or the brand of the product.	
Primary actors: Customer	
Secondary actors: None.	
Preconditions: 1. The user is currently viewing the product catalogue (UC8).	
Main flow: Not required	
Extension use cases: None.	
Inclusion use cases: None.	
Specialisations of: None.	
Postconditions: 1. The system displays only those products that match the filtering criteria.	
Alternative flows: NoMatchingItemFound	

Use case ID: 10	Use case: ViewActivePromotions
------------------------	---------------------------------------

Brief Description: Allows guests and customers to view current advertising campaigns active between specified Start and End dates.
Primary actors: Customer, Guest
Secondary actors: None.
Preconditions: 1. At least one campaign is active based on system time.
Main flow: Not required.
Extension use cases: SelectPromotionCampaign(UC11)
Inclusion use cases: None
Specialisations of: None.
Postconditions: 1. List of active campaigns is displayed.
Alternative flows: NoActivePromotionsAvailable

Use case ID: 11	Use case: SelectPromotionCampaign
Brief Description: Customer selects a specific campaign to see discounted items. The system increments the campaign "hits" counter.	
Primary actors: Customer	
Secondary actors: None.	
Preconditions: 1. Customer is viewing active promotions (UC10).	

Main flow: Not required.
Extension use cases: None.
Inclusion use cases: TrackCustomerEngagement (UC12)
Specialisations of: None.
Postconditions: 1. Discounted items list displayed.
Alternative flows: CampaignExpired

Use case ID: 12	Use case: TrackCustomerEngagement
Brief Description: The system increments the campaign "hits" counter when a customer selects a specific campaign to see discounted items.	
Primary actors: Customer	
Secondary actors: None.	
Preconditions: 2. Customer is viewing active promotions (UC10). 3. Customer selected promotion campaign (UC11).	
Main flow: Not required.	
Extension use cases: None.	
Inclusion use cases: None.	
Postconditions: 1. CAMP_X_counter increased.	

Alternative flows: IncrementSyncError

Use case ID: 13	Use case: AddToCart
Brief Description: Allows the customer to select products and adds quantities from the catalogue to prepare an order.	
Primary actors: Customer.	
Secondary actors: None.	
Preconditions: 1. User is viewing the catalogue.	
Main flow: Not required.	
Extension use cases: None.	
Inclusion use cases: None.	
Specialisations of: None.	
Postconditions: 1. Items are added to the session's shopping cart.	
Alternative flows: InsufficientStockAvailability	

Use case ID: 14	Use case: PlaceOrder
------------------------	-----------------------------

Brief Description: Allows the customer to confirm the items in the shopping cart and completes the purchase by paying with a credit/debit card. The system initiates payment processing and updates the merchant's inventory.
Primary actors: Customer.
Secondary actors: PaymentProcessor, IPOS-CA, EmailService
Preconditions: 1. Products have been selected and added to the cart (UC13).
Main flow: Not required.
Extension use cases: None.
Inclusion use cases: MakePayment (UC15), TransmitOrderIPOS-CA (UC16), SendEmail (UC5).
Specialisations of: None.
Postconditions: 1. The order is confirmed. 2. Payment is made and recorded. 3. Merchant's stock in IPOS-CA is deducted. 4. A confirmation email with tracking details is sent.
Alternative flows: PaymentDeclined TechnicalSyncError

Use case ID: 15	Use case: MakePayment
Brief Description: This use case handles the secure processing of credit or debit card transactions through a third-party payment processor. It ensures that the order amount is authorized and captured.	
Primary actors: Customer	

Secondary actors:
PaymentProcessor
Preconditions:
1. The shopping cart total has been calculated. 2. The user has reached the checkout stage in PlaceOrder(UC14).
Main flow: Not required.
Extension use cases: None
Inclusion use cases: None
Specialisations of: None.
Postconditions:
1. Transaction is successfully processed by the external service payment processor. 2. Payment details (payee name, amount, time, and card info) are stored in the system database.
Alternative flows:
PaymentDeclined
ConnectionTimeout

Use case ID: 16	Use case: TransmitOrderIPOS-CA
Brief Description: Propagates successful online orders to the merchant's local inventory (IPOS-CA) to deduct stock	
Primary actors: Customer (initiated via UC14).	
Secondary actors:	
IPOS-CA.	
Preconditions:	
1. Online payment is confirmed.	

Main flow: Not required.
Extension use cases: None
Inclusion use cases: None
Specialisations of: None.
Postconditions: 1. Local stock availability in IPOS-CA is updated.
Alternative flows: SyncFailureWithIPOS-CA

Use case ID: 17	Use case: TrackOrder
Brief Description: Allows customers to view the current status of their order (Received, Dispatched, Delivered).	
Primary actors: Customer.	
Secondary actors: None.	
Preconditions: 1. An order has been successfully placed.	
Main flow: Not required.	
Extension use cases: None	
Inclusion use cases: None	
Specialisations of: None.	
Postconditions:	

1. Current status of the order is displayed to the user.
Alternative flows: OrderIDNotFound

Use case ID: 18	Use case: ManageCampaigns
Brief Description: MerchantAdmin sets up, modifies, or cancels advertising campaigns with fixed or variable discounts.	
Primary actors: MerchantAdmin	
Secondary actors: None.	
Preconditions: 1. IPOS-PU portal is accessible.	
Main flow: Not required.	
Extension use cases: None.	
Inclusion use cases: None	
Specialisations of: None.	
Postconditions: 1. Campaign details and analytics counters are initialized in the database.	
Alternative flows: InvalidDateRange	

Use case ID: 19	Use case: GenerateReport
Brief Description: Produces a report of all online sales within a specified period.	
Primary actors: MerchantAdmin.	
Secondary actors: None.	
Preconditions: 1. Sales data exists for the selected period.	
Main flow: Not required.	
Extension use cases: PrintReport (UC20)	
Inclusion use cases: None	
Specialisations of: None.	
Postconditions: 1. Report is displayed according to the Appendix 8 layout	
Alternative flows: NoDataFound	

Use case ID: 20	Use case: PrintReport
Brief Description: Prints the report generated in UC19.	
Primary actors: MerchantAdmin.	
Secondary actors: None.	
Preconditions:	

2. Report successfully generated in UC19.
Main flow: Not required.
Extension use cases: None.
Inclusion use cases: None
Specialisations of: None.
Postconditions: 2. Report is printed.
Alternative flows: PrintError

Use case ID: 21	Use case: GenerateSalesReport
Brief Description: Produces a report of all online sales within a specified period, including information of items sold and total revenue.	
Primary actors: MerchantAdmin.	
Secondary actors: None.	
Preconditions: 3. Sales data exists for the selected period.	
Main flow: Not required.	
Extension use cases: PrintReport (UC20)	
Inclusion use cases: None	
Specialisation of: GenerateReport (UC19)	

Postconditions: 3. Report is displayed according to the Appendix 8 layout.
Alternative flows: NoDataFound

Use case ID: 22	Use case: GenerateAdvertisingCampaignReport
Brief Description: Provides details on active campaigns, including information of items sold and total sales per campaign.	
Primary actors: MerchantAdmin	
Secondary actors: None.	
Preconditions: 1. Campaign data is available.	
Main flow: Not required.	
Extension use cases: PrintReport (UC20)	
Inclusion use cases: None	
Specialisation of: GenerateReport (UC19)	
Postconditions: 1. Report is displayed according to the Appendix 9 layout.	
Alternative flows: None	

Use case ID: 23	Use case: GenerateCustomerEngagementReport
------------------------	---

Brief Description: Analyzes campaign success by comparing hits vs. purchases to calculate conversion rates.
Primary actors: MerchantAdmin.
Secondary actors: None.
Preconditions: 1. Campaign activity has been recorded.
Main flow: Not required.
Extension use cases: PrintReport (UC20)
Inclusion use cases: None
Specialisation of: GenerateReport (UC19)
Postconditions: 1. Report is displayed according to the Appendix 10 layout.
Alternative flows: ZeroPurchasesResultingNoConversionRate

Use case ID: 24	Use case: ProvideEmailSendingService
Brief Description: Exposes the IPOS-PU email functionality to IPOS-SA and IPOS-CA subsystems.	
Primary actors: IPOS-SA, IPOS-CA.	
Secondary actors: EmailService	
Preconditions: 1. Request received from an external IPOS subsystem.	
Main flow: Not required.	

Extension use cases: None
Inclusion use cases: SendEmail (UC5)
Specialisations of: None.
Postconditions: 1. External subsystem's email request is processed.
Alternative flows: InvalidRequestFormat.

5.2.1.3 Prioritised List of use cases

Use case ID	Use case name	UC priority	Justification
UC1	ApplyMembership	Essential	Allows guests to register for membership.
UC2	ApplyCommercialMembership	Essential	Allows companies to submit registration data to IPOS-SA for verification.
UC3	ApplicationSubmissionIPOS-SA	Essential	Critical data transmission for merchant onboarding.
UC4	ApplyNon-CommercialMembership	Important	Simple email-based registration for the general public.
UC5	SendEmail	Essential	Core automated notification service (passwords, tracking, receipts).
UC6	Login	Essential	Core security access point for all registered users and admins.
UC7	ChangePassword	Essential	Mandatory security update for first-time logins and user maintenance.
UC8	ViewCatalogue	Essential	Primary interface for browsing products, prices, and availability.

UC9	FilterCatalogue	Desirable	Optional search/filter tool to find specific items in the catalogue.
UC10	ViewActivePromotions	Important	Visibility of time-limited marketing campaigns for customers.
UC11	SelectPromotionCampaign	Important	Interaction with specific promotions to view discounted items.
UC12	TrackCustomerEngagement	Important	Tracking of interactions with specific promotions.
UC13	AddToCart	Essential	Core sales function to select items for purchase.
UC14	PlaceOrder	Essential	Critical checkout process initiating payment and order fulfillment.
UC15	MakePayment	Essential	Secure handling of financial transactions via external processor.
UC16	TransmitOrderIPOS-CA	Essential	Automated inventory synchronization with the pharmacy's local system.
UC17	TrackOrder	Important	Post-purchase service to view delivery status.
UC18	ManageCampaigns	Essential	Admin control over marketing parameters and analytics initialization.
UC19	GenerateReport	Essential	Generation of report on sales.
UC20	PrintReport	Important	Enables reports generated to be printed.
UC21	GenerateSalesReport	Essential	Financial reporting for the merchant (Appendix 8).
UC22	GenerateAdvertisingCampaignReport	Important	Analytical report on campaign performance (Appendix 9).
UC23	GenerateCustomerEngagementReport	Essential	Key conversion analytics (Hits vs. Purchases) for business strategy.

UC24	ProvideEmailSendingService	Important	Cross-system integration providing email features to SA/CA subsystems.
------	----------------------------	-----------	--

The list below prioritises the use cases included in the use case model according to their importance for the IPOS-PU. CSR management will use the assigned priorities to decide on the order of implementing the functionality specified in the use cases according to the resources available: the **essential** use cases will be implemented first, then the use cases ranked as **important** will follow. The use cases ranked as **good-to-have** will be implemented last provided, the resources (manpower and time) are available to complete their implementation.

6. ANTICIPATED EVOLUTION OF SYSTEM REQUIREMENTS

We expect that IPOS-PU will be deployed early in the operational life of the InfoPharma Ordering System, allowing merchants to begin selling directly to the public and to run promotional campaigns. As real customers and merchants begin using the platform, their feedback and usage patterns will inform future enhancements. The areas of evolution are outlined below.

6.1 Customer Registration and Verification

During the initial release, IPOS-PU will support basic customer registration with minimal validation of personal details. An email confirmation will be implemented, but verification of mobile numbers or identity checks will not be required. For future versions, we will anticipate the introduction of stronger verification mechanisms, such as:

- ❖ Optional or mandatory two-factor authentication for customer logins
- ❖ Integration with third-party identity verification services for age-restricted pharmaceutical products
- ❖ Enhanced validation of email and phone numbers for improved security and fraud prevention

Changes may require updates to the registration workflow and additional interfaces with external verification providers.

6.2 Payment Processing

The initial specification assumes a simplified payment workflow, with payments recorded internally and no integration with external payment gateways. Future enhancements may include:

- ❖ Integration with real payment service providers (e.g. PayPal)
- ❖ Support for multiple payment methods, including digital wallets
- ❖ Automated handling of refunds, chargebacks and failed transactions

Changes will require modifications to the payment subsystem and additional security considerations.

6.3 Promotional Campaigns

First versions of IPOS-PU will allow merchants to create basic promotional campaigns with fixed discounts and simple start/end dates. Anticipations like merchants will request more sophisticated marketing capabilities, such as:

- ❖ Targeted promotions based on customer purchase history
- ❖ Support for voucher codes, tiered discounts or limited-quantity offers
- ❖ Testing of campaign variants
- ❖ Automated scheduling and campaign templates

Enhancements like these will require extensions to the campaign management module and the reporting subsystem.

6.4 Customer Engagement Tracking

Initially, IPOS-PU will generate standard reports on campaign engagement, including views, clicks and completed purchases. Future versions may expand this functionality to include:

- ❖ Real-time analytics dashboards
- ❖ Integration with third-party analytics platforms
- ❖ Tracking of customer behaviour across multiple sessions and devices
- ❖ Predictive analytics to estimate campaign performance

This evolution will require additional data collection, storage and processing capabilities.

6.5 Catalogue and Inventory Synchronisation

Currently, IPOS-PU relies on periodic synchronisation with IPOS-CA and IPOS-SA for product availability and pricing. As the system progresses, there should be expectations toward a shift in:

- ❖ Real-time stock updates to prevent overselling
- ❖ Automated alerts to merchants when online demand exceeds stock levels
- ❖ Support for pre-orders or back-orders

Changes will require tighter coupling between subsystems and more robust interface definitions.

6.6 Customer Loyalty and Retention

Although not included in the initial release, there will be anticipation towards merchants that may request features to encourage repeat purchases, such as:

- ❖ Loyalty points or reward programme
- ❖ Personalised recommendations
- ❖ Customer segmentation for targeted marketing

Implementing such features will require additional data models and business logic within the IPOS-PU.

7. IPOS HIGH – LEVEL DESIGN

This section defines the high-level design of the entire

7.1 IPOS – SA

This subsystem should allow InfoPharma Ltd. to maintain an electronic catalogue of pharmaceutical products and control stock availability.

7.2 IPOS – CA

This subsystem is a desktop application which implements a combination of features allowing the merchant to access IPOS-SA-ORD, and a set of additional features to automate the operation of the merchant in dealing with their own customers.

7.3 IPOS – PU

This subsystem offers an online portal which a merchant may use to sell goods to members of the public online.

7.4 Software architecture

The architecture of IPOS-PU as a UML component diagram, developed with Visual Paradigm, Professional Edition, is shown on Figure 2.

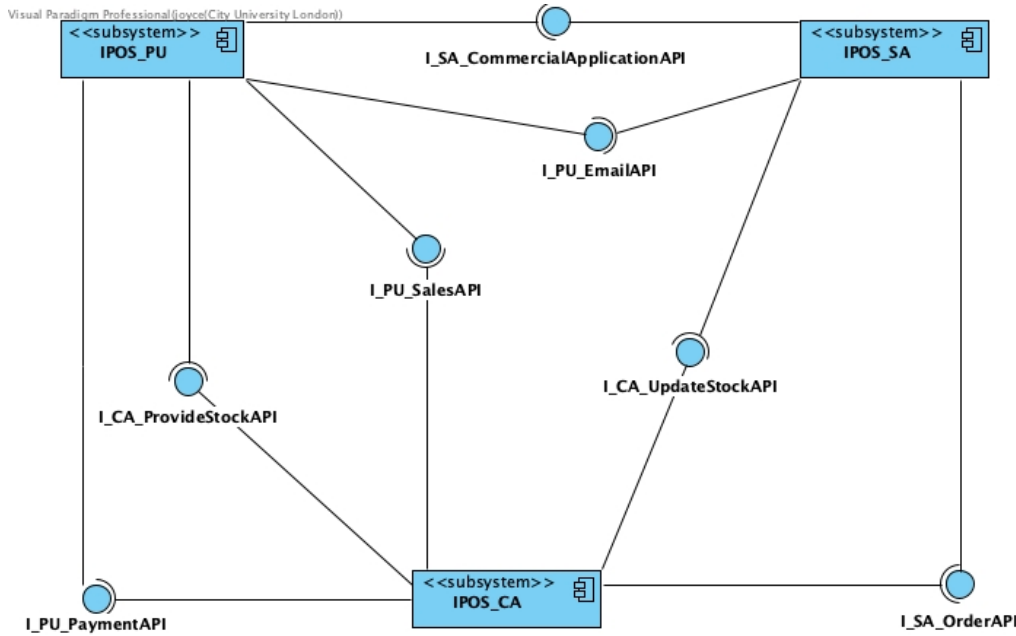


Figure 2. IPOS architecture as a UML component diagram. Each subsystem is modelled as a component with stereotype `<<subsystem>>`. The communication between the subsystems is achieved via a set of assembly connectors (i.e., a pair of “provided” and “required” interfaces referring to the API offered by the respective subsystems. *I_SA_CommercialApplicationAPI*, *I_SA_OrderAPI*, *I_CA_ProvideStockAPI*, *I_CA_UpdateStockAPI*, *I_PU_SalesAPI*, *I_PU_EmailAPI*, and *I_PU_PaymentAPI* are provided by *IPOS_SA*, *IPOS_CA* and *IPOS_PU*, respectively and the APIs are used to access the functionality offered by one of the subsystems by the other two subsystems.

Figure 2 hides the API details. Using the “presentation options” of the Visual Paradigm, one can switch to “class” notations and disclose full interface details^[1], as is illustrated on **Figure 3**.

^[1] Switching the appearance of interfaces between the lollipop syntax (*Figure 2*) and the “class” syntax (*Figure 3*) can be done via the “Presentation Options” applied to interfaces. The default presentation option is the lollipop notation, which can be changed to “class” notation, typical for a design class diagram, to show the interface as a class compartment with all abstract methods defined for the interface.

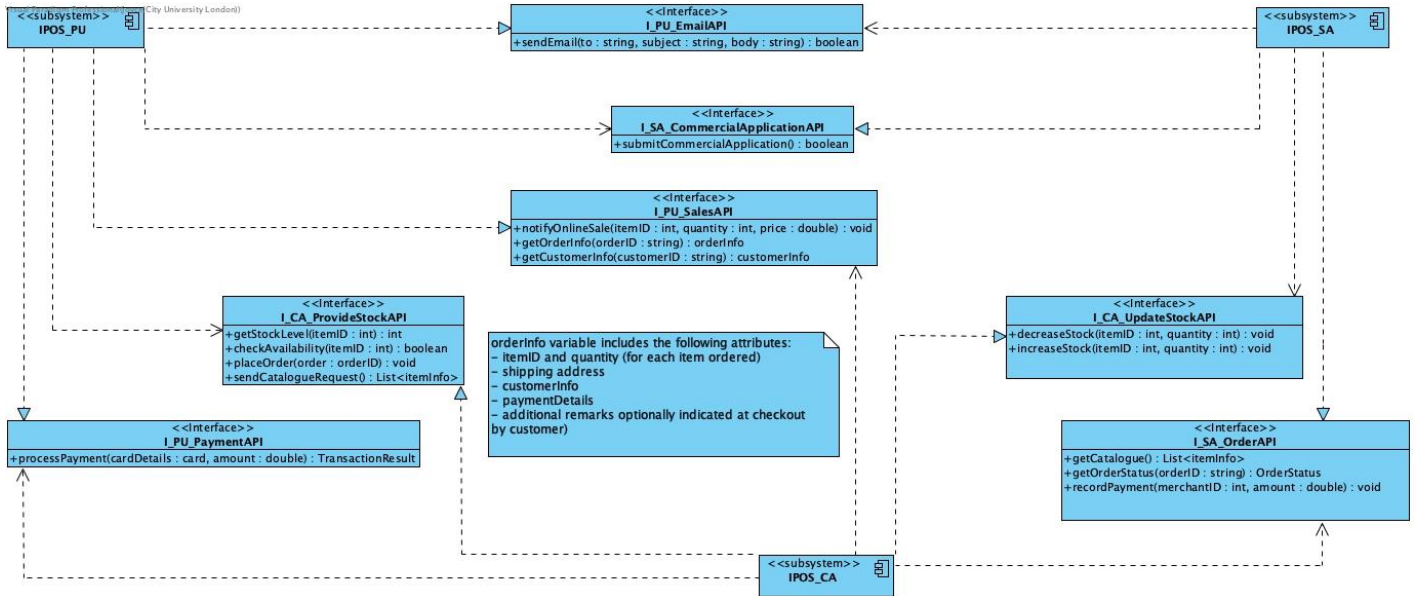


Figure 3. Now the methods defined in interfaces are explicitly shown with their full signatures. These are plausible public methods via which the functionality of respective subsystem can be accessed. Note that changing the presentation of interfaces, changed the relationships of the interfaces, too: the subsystems responsible to implement the interface has a “realisation” relationship with it, which implies that the respective subsystem will contain an implementation class realising the interface. The subsystems using the API (i.e. for which the API is a “required” interface) now would have a dependency relationship with the interface (<<usage>> icon in Visual Paradigm should be used in the construction of the component diagram).

The IPOS architecture includes three subsystems and shows the interfaces necessary for these subsystems to interact with each other.

7.4. DESIGN DIAGRAM OF SUBSYSTEM IPOS-PU

Next, we elaborate the high-level design of IPOS-PU and provide details on the interfaces that this subsystem needs, and on the implementation of the subsystem’s required interface.

The class diagram of IPOS-PU subsystem is shown in *Figure 4*.

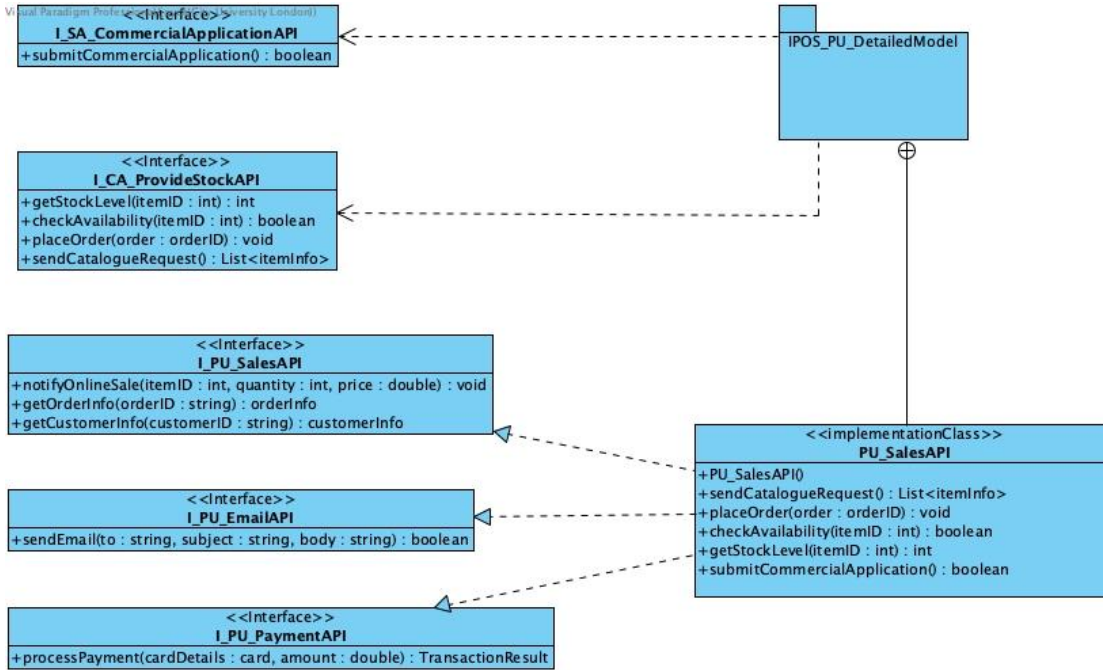


Figure 4. High-Level design of IPOS_PU.

The diagram shows the interfaces that IPOS_PU depends on (which already appeared in full detail in Figure 3), and a minimal version^[1] of the implementation class PU_SalesAPI of the interface I_PU_SalesAPI, I_PU_EmailAPI, and I_PU_PaymentAPI together with the realisation relationship between the provided interface and the implementation class. The diagram also makes it clear that the implementation class is part of IPOS_PU_DetailedModel package: the containment relationship between the package and the implementation class shows that the implementation class belongs to the package. At this stage, the package IPOS_PU_DetailedModel is just a place holder. Its content – classes, interfaces, enumerations, etc., which together will deliver the functionality of IPOS_PU subsystem – will be developed later, as part of detailed design and implementation of IPOS, which are yet to take place. The diagram also shows the required interfaces I_SA_CommercialApplicationAPI and I_CA_ProvideStockAPI, which we saw in Figure 3. The dependency relationship between IPOS_PU_DetailedModel package and the two required interfaces indicate that some of the classes in the package IPOS_PU_DetailedModel may/will use the required interfaces (i.e. will be making calls of the methods defined in the interfaces) to access the functionality of the respective subsystems, which in turn will be providing an implementation for the methods of the required interfaces, as required.

[1] Minimalistic because the implementation class only offers realisations of all abstract methods defined in the interface + a constructor. In practice, implementation classes may contain other methods, not defined in the interface, with visibility, scope and parameters as required by the specific needs. The implementation classes may also have attributes with private/protected scope, etc., as required. These details are likely to emerge during the detailed design/implementation of IPOS subsystems and are unlikely to be known at this early stage of development.

7.5 Source code (Java) for the interfaces of IPOS-PU

Now that we have defined all interfaces and the implementation class PU_SalesAPI, source code could be derived (e.g. Java source code) for the interfaces and the implementation classes shown in Figure 4. This can be done with Visual Paradigm. The complete Java code of the interfaces and a skeleton of the implementation class are shown below generated by Visual Paradigm is provided below:

```
package IPOS;

public interface I_CA_ProvideStockAPI {

    /**
     *
     * @param itemID
     */
    int getStockLevel(int itemID);

    /**
     *
     * @param itemID
     */
    boolean checkAvailability(int itemID);

    /**
     *
     * @param order
     */
    void placeOrder(orderID order);

    List<itemInfo> sendCatalogueRequest();

}

package IPOS;

public interface I_PU_EmailAPI {

    /**
     *
     * @param to
     * @param subject
     * @param body
     */
    boolean sendEmail(string to, string subject, string body);

}
```

```

package IPOS;

public interface I_PU_PaymentAPI {

    /**
     *
     * @param cardDetails
     * @param amount
     */
    TransactionResult processPayment(card cardDetails, double amount);

}

```

```

package IPOS;

public interface I_PU_SalesAPI {

    /**
     *
     * @param itemID
     * @param quantity
     * @param price
     */
    void notifyOnlineSale(int itemID, int quantity, double price);

    /**
     *
     * @param orderID
     */
    orderInfo getOrderInfo(string orderID);

    /**
     *
     * @param customerID
     */
    customerInfo getCustomerInfo(string customerID);

}

```

```

public interface I_SA_CommercialApplicationAPI {

    boolean submitCommercialApplication();

}

```

```

package IPOS_PU_DetailedModel;

import IPOS.*;

public class PU_SalesAPI implements I_PU_EmailAPI, I_PU_PaymentAPI, I_PU_SalesAPI {

    public PU_SalesAPI() {
        // TODO - implement PU_SalesAPI.PU_SalesAPI
        throw new UnsupportedOperationException();
    }

    public List sendCatalogueRequest() {
        // TODO - implement PU_SalesAPI.sendCatalogueRequest
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param order
     */
    public void placeOrder(orderID order) {
        // TODO - implement PU_SalesAPI.placeOrder
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param itemID
     */
    public boolean checkAvailability(int itemID) {
        // TODO - implement PU_SalesAPI.checkAvailability
        throw new UnsupportedOperationException();
    }

    /**
     *
     * @param itemID
     */
    public int getStockLevel(int itemID) {
        // TODO - implement PU_SalesAPI.getStockLevel
        throw new UnsupportedOperationException();
    }

    public boolean submitCommercialApplication() {
        // TODO - implement PU_SalesAPI.submitCommercialApplication
        throw new UnsupportedOperationException();
    }

}

```

7.6 Testing the communications between subsystems 13

Now that we have defined the interfaces necessary for communication between the subsystems of InfoPharma and generated Java code for all of them, we develop tests for inter-subsystem communication, i.e. of the respective implementation classes⁸.

Here we sketch tests for the single implementation class without committing to a particular testing platform to run the tests (e.g. junit).

Method	Test ID	Parameter Values	Comment	Expected Outcome
<code>checkAvailability(itemID:int):boolean</code>	1	<code>itemID = 101</code>	Item exists and has stock available	Success (the boolean returned is "true"). Item confirmed as available.
<code>checkAvailability(itemID:int):boolean</code>	2	<code>itemID = 101</code>	Item exists but stock level is 0	Failure (the boolean returned is "false"). Item reported as unavailable.
<code>checkAvailability(itemID:int):boolean</code>	3	<code>itemID = 999</code>	Item does not exist in the catalogue	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>checkAvailability(itemID:int):boolean</code>	4	<code>itemID = -1</code>	Negative itemID supplied	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>getStockLevel(itemID:int):int</code>	5	<code>itemID = 101</code>	Item exists with stock greater than zero	Success. Returns the correct stock count (e.g. 50).

<code>getStockLevel(itemID:int):int</code>	6	<code>itemID = 101</code>	Item exists but stock is zero	Success. Returns 0.
<code>getStockLevel(itemID:int):int</code>	7	<code>itemID = 999</code>	Item does not exist in the catalogue	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>getStockLevel(itemID:int):int</code>	8	<code>itemID = -5</code>	Negative <code>itemID</code> supplied	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>placeOrder(order:orderID):void</code>	9	<code>order = valid orderID with existing items and sufficient stock</code>	Valid order submitted	Success. Order placed. Stock decremented accordingly. No exception thrown.
<code>placeOrder(order:orderID):void</code>	10	<code>order contains itemID = 999</code>	Item in order does not exist in catalogue	Failure. <code>IllegalArgumentException</code> thrown. No order placed.
<code>placeOrder(order:orderID):void</code>	11	<code>order = null</code>	Null order object supplied	Failure. <code>NullPointerException</code> thrown. No order placed.
<code>placeOrder(order:orderID):void</code>	12	<code>order quantity exceeds available stock</code>	Requested quantity is greater than current stock level	Failure. <code>IllegalStateException</code> thrown. No order placed.
<code>sendCatalogueRequest():List<itemInfo></code>	13	(no parameters)	Catalogue contains one or more items	Success. Returns a non-empty <code>List<itemInfo></code>

				with correct item details.
<code>sendCatalogueRequest():List<itemInfo></code>	14	(no parameters)	Catalogue is empty	Success. Returns an empty list. No exception thrown.
<code>sendEmail(to:string, subject:string, body:string):boolean</code>	15	<code>to = "customer@example.com"subject = "Order Confirmed"body = "Your order has been placed."</code>	Valid email address, subject and body provided	Success (the boolean returned is "true"). Email sent successfully.
<code>sendEmail(to:string, subject:string, body:string):boolean</code>	16	<code>to = "not-an-email"subject = "Test"body = "Test body"</code>	Invalid email address format provided	Failure (the boolean returned is "false"). No email sent.
<code>sendEmail(to:string, subject:string, body:string):boolean</code>	17	<code>to = ""subject = "Test"body = "Test body"</code>	Empty to field supplied	Failure. IllegalArgumentException thrown. No email sent.
<code>sendEmail(to:string, subject:string, body:string):boolean</code>	18	<code>to = "customer@example.com"subject = "Test"body = ""</code>	Empty body supplied	Failure (the boolean returned is "false"). No email sent.
<code>processPayment(cardDetails:card, amount:double):TransactionResult</code>	19	<code>cardDetails = valid cardamount = 99.99</code>	Valid card details and positive amount supplied	Success. Returns TransactionResult with status "approved". Payment recorded.
<code>processPayment(cardDetails:card, amount:double):TransactionResult</code>	20	<code>cardDetails = card with invalid</code>	Invalid card number supplied	Failure. Returns TransactionResult with status

		numberamount = 99.99		"declined". No payment taken.
processPayment(cardDetails:card, amount:double):TransactionResult	21	cardDetails = nullamount = 99.99	Null card object supplied	Failure. NullPointerException thrown. No payment attempted.
processPayment(cardDetails:card, amount:double):TransactionResult	22	cardDetails = validcardamount = -50.00	Negative amount supplied	Failure. IllegalArgumentException thrown. No payment attempted.
processPayment(cardDetails:card, amount:double):TransactionResult	23	cardDetails = valid card with insufficient fundsamount = 99.99	Card has insufficient funds to cover the amount	Failure. Returns TransactionResult with status "insufficient funds". No payment taken.
notifyOnlineSale(itemID:int, quantity:int, price:double):void	24	itemID = 101quantity = 3price = 29.99	Valid item, positive quantity and price	Success. Sale notification sent. No exception thrown.
notifyOnlineSale(itemID:int, quantity:int, price:double):void	25	itemID = 999quantity = 3price = 29.99	Item does not exist in the catalogue	Failure. IllegalArgumentException thrown. No notification sent.
notifyOnlineSale(itemID:int, quantity:int, price:double):void	26	itemID = 101quantity = -1price = 29.99	Negative quantity supplied	Failure. IllegalArgumentException thrown. No notification sent.
notifyOnlineSale(itemID:int, quantity:int, price:double):void	27	itemID = 101quantity = 3price = -5.00	Negative price supplied	Failure. IllegalArgumentException thrown. No notification sent.

<code>getOrderInfo(orderID:string):orderInfo</code>	28	<code>orderID = "ORD-001"</code>	Order exists in the system	Success. Returns correct <code>orderInfo</code> object with matching order details.
<code>getOrderInfo(orderID:string):orderInfo</code>	29	<code>orderID = "ORD-999"</code>	Order does not exist in the system	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>getOrderInfo(orderID:string):orderInfo</code>	30	<code>orderID = null</code>	Null <code>orderID</code> supplied	Failure. <code>NullPointerException</code> thrown. No result returned.
<code>getCustomerInfo(customerID:string):customerInfo</code>	31	<code>customerID = "CUST-001"</code>	Customer exists in the system	Success. Returns correct <code>customerInfo</code> object with matching customer details.
<code>getCustomerInfo(customerID:string):customerInfo</code>	32	<code>customerID = "CUST-999"</code>	Customer does not exist in the system	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>getCustomerInfo(customerID:string):customerInfo</code>	33	<code>customerID = null</code>	Null <code>customerID</code> supplied	Failure. <code>NullPointerException</code> thrown. No result returned.
<code>submitCommercialApplication():boolean</code>	34	(no parameters)	Application data is valid and complete	Success (the boolean returned is "true"). Application submitted successfully.

submitCommercialApplication():boolean	35	(no parameters)	Application has already been submitted previously	Failure (the boolean returned is "false"). Duplicate submission rejected.
submitCommercialApplication() : boolean	36	No parameters	A required field is missing.	Failure (the boolean returned is "false"). Application not submitted.
checkApplicationStatus(applicationID:string)	37	applicationID = "IP1001"	Application exists and status is Approved	Success. Status "Approved" returned.
checkApplicationStatus(applicationID:string)	38	applicationID = "IP1002"	Application exists and status is Pending	Success. Status "Pending" returned.
checkApplicationStatus(applicationID:string)	39	applicationID = "IP9999"	Application does not exist	Failure. IllegalArgumentException thrown. No result returned.
checkApplicationStatus(applicationID:string)	40	applicationID = ""	Empty applicationID supplied	Failure. IllegalArgumentException thrown. No result returned.
getCatalogue():List<Item>	41	No parameters	Catalogue contains multiple items	Success. A non-empty List<Item> is returned.

<code>getCatalogue():List<Item></code>	42	No parameters	Catalogue is empty	Success. An empty <code>List<Item></code> is returned.
<code>getCatalogue():List<Item></code>	43	No parameters	Database connection failure	Failure. Exception thrown. No list returned.
<code>getOrderStatus():OrderStatus</code>	44	Order status = "DELIVERED"	Valid order	Success. <code>OrderStatus</code> "DELIVERED" returned.
<code>getOrderStatus():OrderStatus</code>	45	Order status = "PROCESSING"	Valid order	Success. <code>OrderStatus</code> "PROCESSING" returned.
<code>getOrderStatus():OrderStatus</code>	46	Order does not exist	Invalid order reference	Failure. <code>IllegalArgumentException</code> thrown. No result returned.
<code>recordPayment(merchantID:int, amount:double)</code>	47	merchantID = 1001, amount = 500.00	Valid merchant and valid payment amount	Success. Payment recorded successfully. No value returned.
<code>recordPayment(merchantID:int, amount:double)</code>	48	merchantID = 1001, amount = -50.00	Negative payment amount supplied	Failure. <code>IllegalArgumentException</code> thrown. Payment not recorded.

<code>recordPayment(merchantID:int, amount:double)</code>	49	<code>merchantID = 9999, amount = 250.00</code>	Merchant does not exist	Failure. <code>IllegalArgumentException</code> thrown. Payment not recorded.
<code>recordPayment(merchantID:int, amount:double)</code>	50	<code>merchantID = 1001, amount = 0.00</code>	Zero payment amount supplied	Failure. <code>IllegalArgumentException</code> thrown. Payment not recorded.

8. APPENDCES

8.1 Initial Statement of requirements by the client

IPOS: INFOPHARMA ORDERING SYSTEM

Your company has been invited to develop an “IT system to assist with the distribution of pharmaceutical goods” for InfoPharma Ltd, a major pharmaceutical distributor. InfoPharma intend to optimise their interaction with their clients, retail merchants (individuals running own chemist shop and large retailers with chains of stores, e.g. Boots and Superdrugs), who sell drugs and other pharmaceutical goods to the members of the public. The new system will also allow small retailers to sell pharmaceutical goods online.

--Currently, InfoPharma communicate with merchants by distributing a catalogue of goods available to the interested parties. The catalogue is printed monthly, but in case of major changes of prices special editions can also be printed between the regular updates. Orders are taken by using conventional means such as phone and more recently – e-mail.

Maintaining this way of interacting with the client base is not justified any longer since most of the retailers are already online and have converted parts of their business to ‘paperless’ technology. Deploying a B2B suite of software applications seems long overdue. InfoPharma want a smooth transition from the current catalogue-based operation to operation relying on a new software order tracking system (InfoPharma Ordering System, IPOS). They would like to have in place for a few months the current operation (catalogue and orders by phone/email) together with IPOS which would allow retailers to have access to the most up to date catalogue and easy ordering from a desktop PC. In addition, in order to motivate the clients (mainly the pharmacy shop owners) to switch to IPOS quickly, they would like IPOS to include extensive functionality which can be useful to the retailers in their work with their customers: controlling the local stock availability in shops, maintaining customers database and ability to provide discounts to regular customers. IPOS will also allow the shop owner to offer their products online.

Centre for Software
Reliability

InfoPharma Ltd. may decide to offer your company the development of the **production** version of IPOS and a maintenance contract for it, if you do a good job with the **prototype** of IPOS.

Your consultant has won the contract from InfoPharma's head office to assess their requirements and produce a project plan for the implementation of one system for one Pharmacy. Your team has been tasked with doing the job. She/he asks you to set up an appointment for an interview with Mr. Lancaster, the founder and CEO at InfoPharma Ltd. Mr Lancaster has agreed to provide clarification on the requirements for the new system.

If the project plan, the accompanying schedule and the requirements documents are accurate, and the cost is acceptable you will be invited to complete the contract and implement the prototype.

Initial Requirement

The InfoPharma Ordering System (IPOS) will keep track of the orders of pharmaceutical goods made by merchants to InfoPharma Ltd and will assist the merchants' sales to consumers.

IPOS includes **three** subsystems:

- i) A server application (IPOS-SA), which in the fully functional production version will be administered by the IT department of InfoPharma Ltd.
- ii) A client application (IPOS-CA), a copy of which will be run by the merchant on a desktop PC in the merchant's Pharmacy shop.
- iii) An online platform (IPOS - PU) to be used by a merchant for direct sales of pharmaceutical goods to the public. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

In the prototype of IPOS the three subsystems should be implemented as separate desktop applications able to interact with one another as necessary to meet the requirements. For instance, online sales (via IPOS - PU) should be linked to the inventory of the merchant: successful online purchases should be deducted from the stock of goods available at merchant's shop. Likewise, orders from IPOS-SA and delivered to a merchant, should be easy to record in IPOS-CA and increase the merchant's stock. Finally, promotion campaigns run via IPOS – PU may affect the cost of selling goods online (at reduced price), but not the cost of the same goods if sold in the pharmacy shops.

8.2 Hardware (minimal and recommended configuration)

8.3 Minimal hardware configuration

The minimal hardware configuration, recommended for IPOS development consists of a desktop/laptop PC or MacOS, Dual Core CPU (e.g. i3 – 8100) 4GB of RAM at least 30 GB of free disk space for required programs and database servers. A display of a resolution at or above 1280x720.

8.4 Recommended hardware configuration

The recommended hardware configuration envisaged for development should consist of a Windows laptop/desktop PC or MacOS, Quad Core CPU (e.g. AMD Ryzen 3 3100) 8GB of DDR4/5 RAM at least 30 GB of free disk space (SSD-solid state drive) for required programs and database servers. A display of a resolution at or above 1920 x 1080.

9. INDEX